VIT-AP UNIVERSITY, AMARAVATI

School of Computer Science and Engineering

ACADEMIC PROJECT REPORT

Internal Assessment – III

STUDENT TASK & ASSIGNMENT MANAGER

A Decoupled 3-Tier Academic Workflow System Deployed on AWS

Course Title:	Amazon Web Services
Course Code:	CSE2025
Slot:	A1 + TA1
Faculty Guide:	**Prof. Thota Ramathulasi**
Student Name:	**Harshraj Santosh Nago**
Registration No:	**24BCA7664**
Academic Year:	2026 – 2027

LIVE DEPLOYED APPLICATION URL (AWS S3)

http://student-task-manager-harshraj.s3-website.ap-south-1.amazonaws.com/

1. ABSTRACT

In modern higher education institutions, undergraduate students manage coursework across multiple subjects simultaneously, often leading to scattered submission schedules, missed deadlines, and unorganized academic workloads. The **Student Task & Assignment Manager** is an end-to-end cloud-native web application designed to centralize academic deadlines, track submission velocity, and provide priority-driven schedules.

To comply with industry-standard cloud engineering practices, the application is engineered following the **AWS 3-Tier Decoupled Architecture**:

- **Presentation Tier:** Built using React 18 and Vite, compiled into static single-page application (SPA) bundles and hosted serverless on **Amazon S3** with static website hosting.
- **Application Tier:** Developed in **Java 17 and Spring Boot 3**, deployed on an **Amazon EC2 Ubuntu instance**, and configured as an auto-restarting background service via a native Linux `systemd` daemon.
- **Database Tier:** Powered by **Amazon RDS for PostgreSQL** (`db.t4g.micro`), isolated via AWS Security Groups for secure relational persistence.

The complete project is publicly deployed, active 24/7 in the `ap-south-1` (Mumbai) region, and operates entirely within the **AWS Free Tier** threshold ($0.00 operational cost).

2. PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Students in engineering disciplines routinely handle 5 to 7 courses per semester, each with separate deadlines, lab rubrics, and submission channels. Without a unified system, students face:

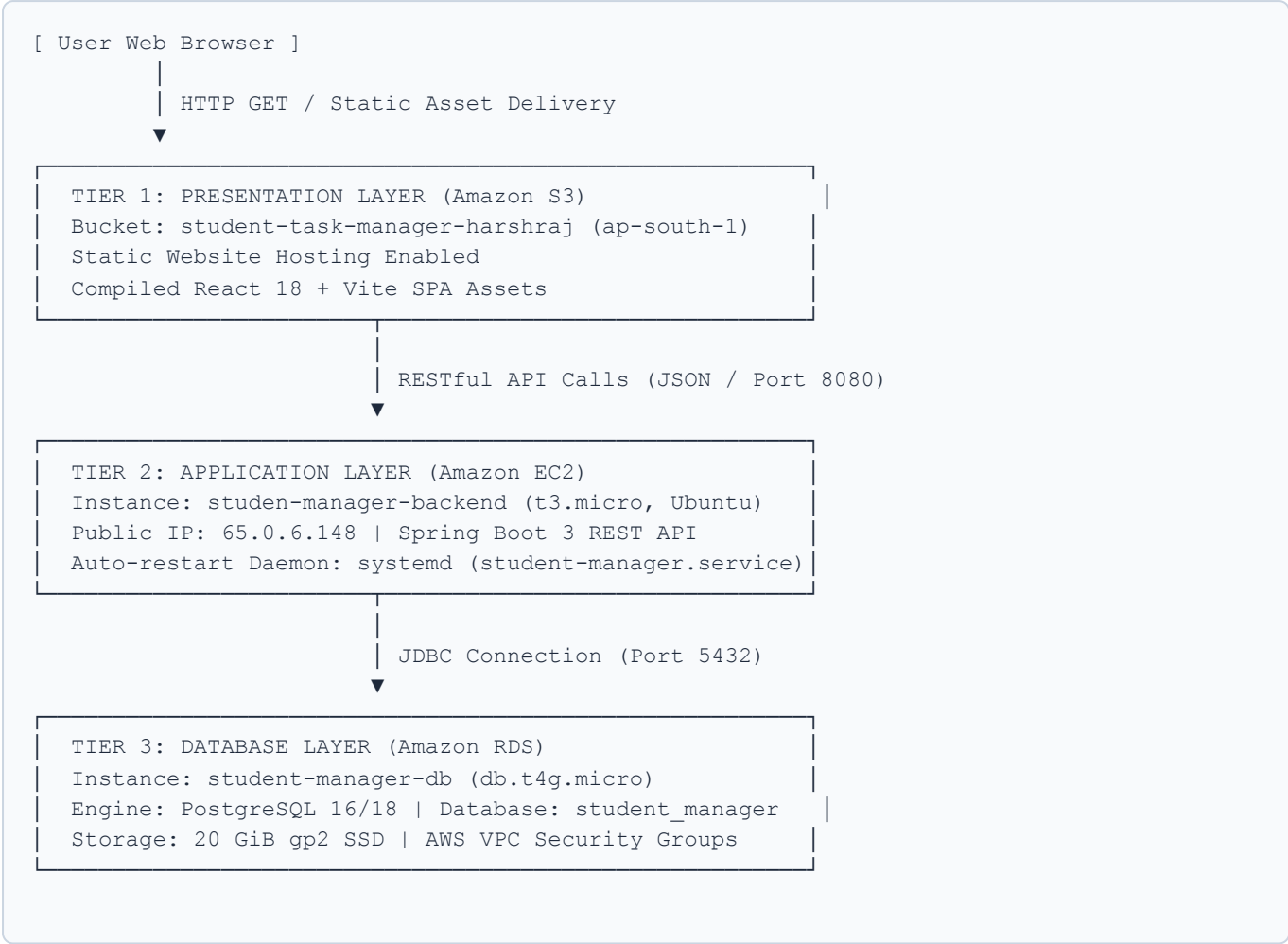
1. **Missed Deadlines:** Inadequate visibility into due dates results in academic penalties.
2. **Fragmented Communication:** Instructions are lost in emails, chat groups, and portal announcements.
3. **Lack of Progress Metrics:** Absence of visual progress indicators prevents timely workload prioritization.

2.2 Project Objectives

- **Decoupled Architecture:** Enforce strict physical and logical separation across static assets (S3), application compute (EC2), and relational storage (RDS).
- **High Availability:** Implement Linux system services to ensure the backend automatically recovers from instance restarts.
- **Managed Cloud Persistence:** Utilize Amazon RDS PostgreSQL to ensure ACID compliance and point-in-time recovery rather than unmanaged local storage.
- **Zero-Cost Operational Sizing:** Optimize all compute, storage, and networking configurations to remain within the AWS Free Tier.

3. SYSTEM ARCHITECTURE (3-TIER CLOUD MODEL)

The system employs a 3-tier decoupled cloud topology. Static presentation delivery is separated from business logic processing and database persistence, preventing cascading failures and optimizing cloud resources:



4. STORAGE & PRESENTATION TIER (AMAZON S3)

The presentation layer is hosted on **Amazon Simple Storage Service (S3)** in the `ap-south-1` (Mumbai) region. S3 serves pre-compiled production HTML, JavaScript, and CSS bundles directly to client browsers without utilizing EC2 processing cycles.

4.1 S3 Bucket Asset Management

The bucket `student-task-manager-harshraj` holds the distribution output from `npm run build`, including `index.html` and the compiled `assets/` directory.

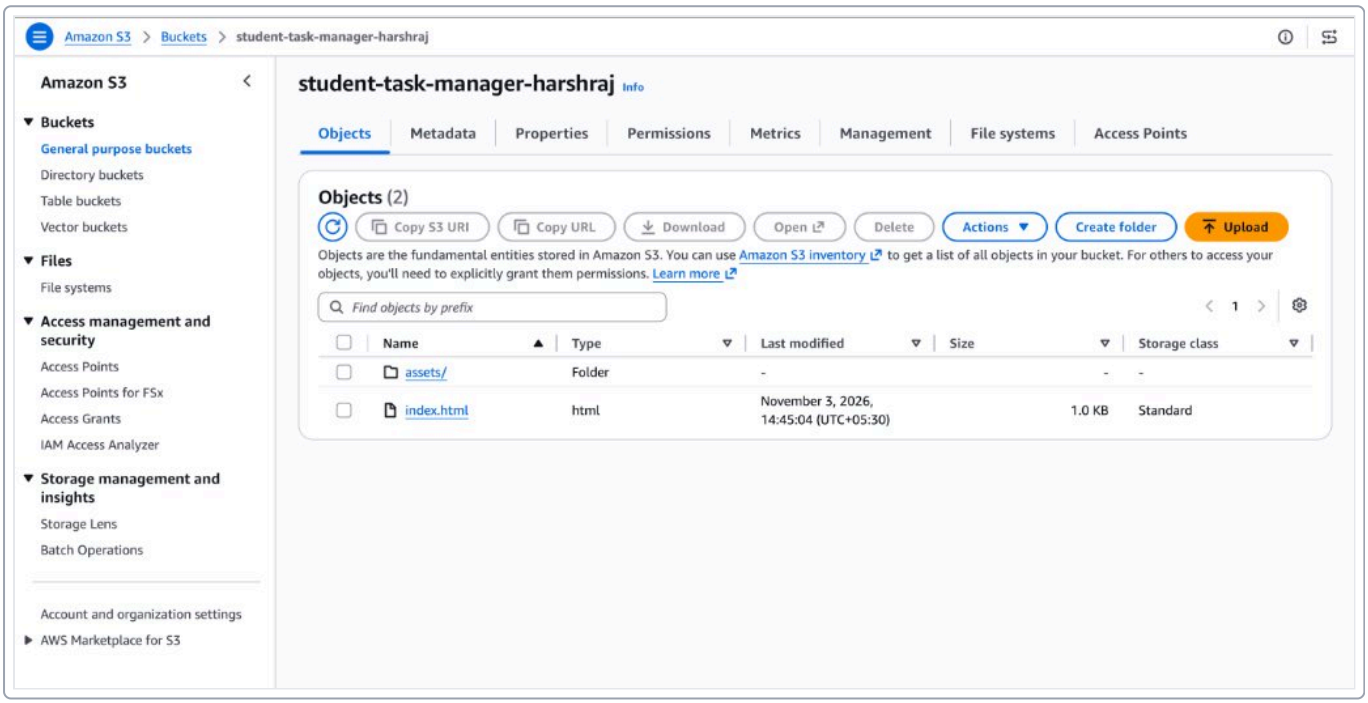


Figure 4.1: Amazon S3 Bucket Objects (index.html and assets/ directory in ap-south-1).

4.2 Static Website Hosting Configuration

Static website hosting was enabled on the bucket, routing both the index and error documents to `index.html` to facilitate client-side React routing.

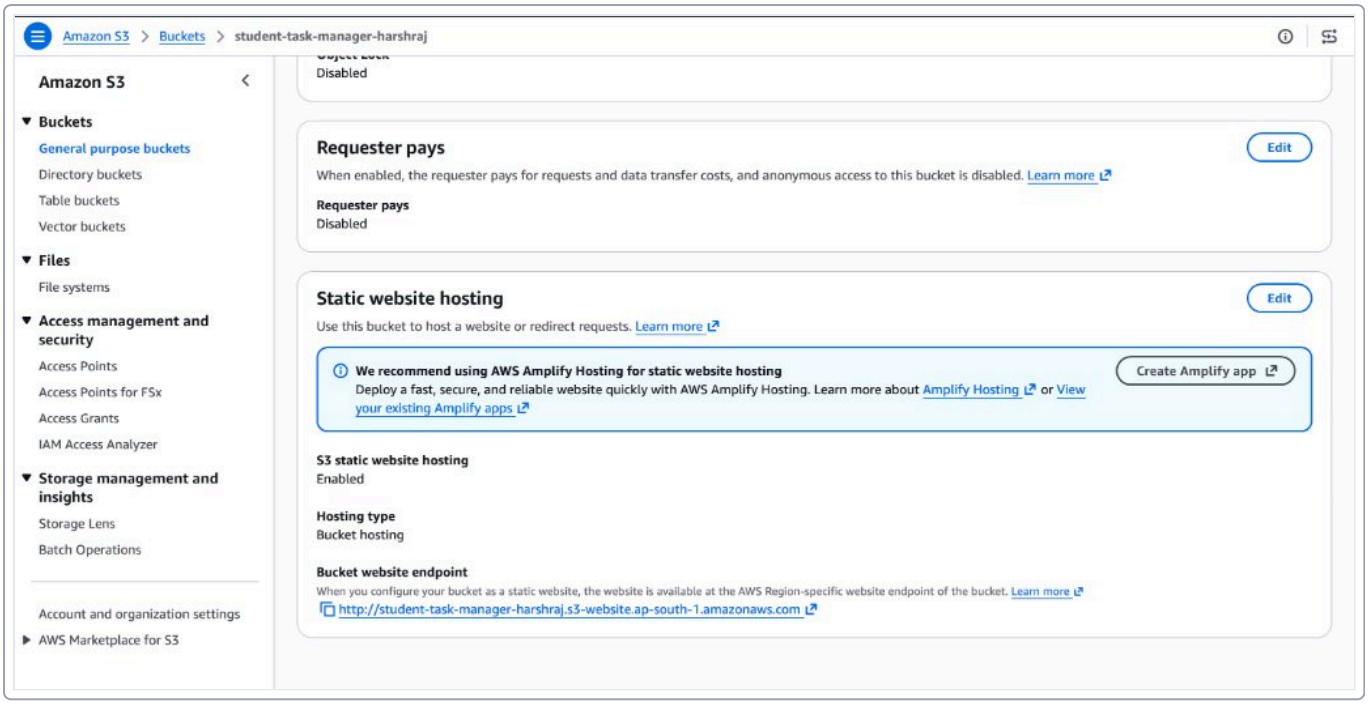


Figure 4.2: Amazon S3 Static Website Hosting Enabled with Live Bucket Endpoint.

4.3 Public Access Bucket Policy

A public read policy was applied to grant read permissions (`s3:GetObject`) to anonymous web traffic while protecting write and bucket management actions:

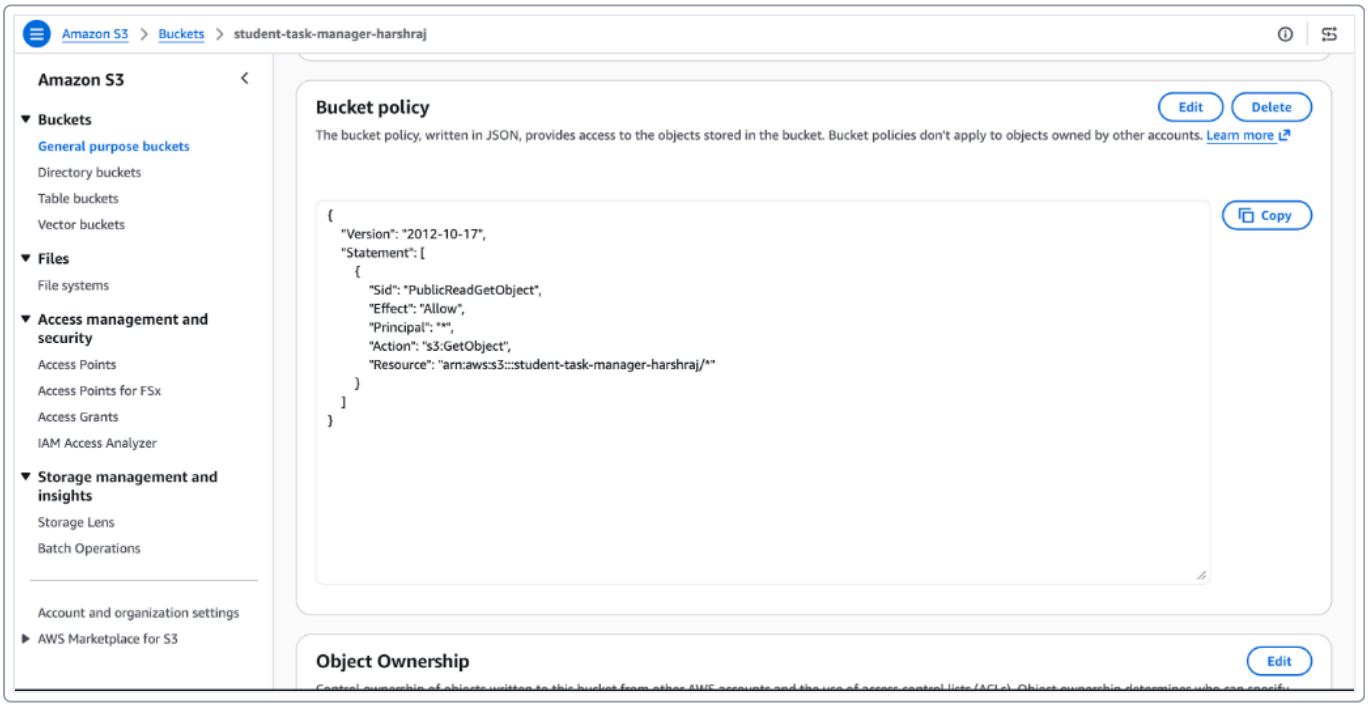


Figure 4.3: JSON Bucket Policy Granting Public Read Access to S3 Assets.

5. APPLICATION & COMPUTE TIER (AMAZON EC2)

The middle tier is responsible for handling REST API requests, data validation, business logic, and database transactions via Java 17 and Spring Boot 3.

5.1 Virtual Server Provisioning

An Amazon EC2 instance was launched with Ubuntu 24.04 LTS to serve as the application host:

- **Instance Name:** `studen-manager-backend`
- **Instance ID:** `i-013eca07c52b74cc1`
- **Instance Type:** `t3.micro` (2 vCPUs, 1.0 GiB RAM)
- **Public IPv4 Address:** `65.0.6.148`
- **Private IPv4 Address:** `172.31.14.7`
- **Availability Zone:** `ap-south-1b`

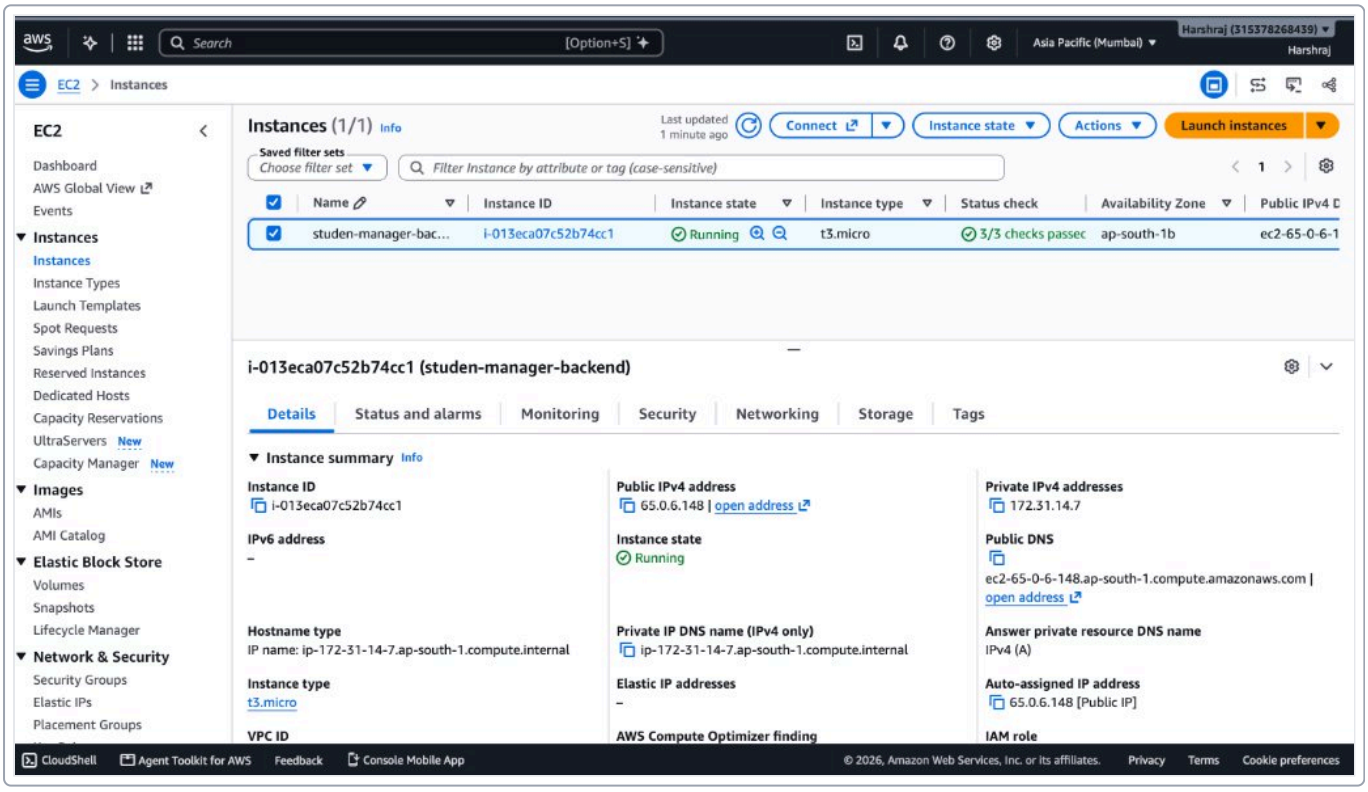


Figure 5.1: Amazon EC2 Instance Overview (studen-manager-backend, Public IP: 65.0.6.148).

5.2 Security Group Firewall Rules

Inbound network traffic to the instance is governed by Security Group `launch-wizard-1` (`sg-050d7e75e205e642e`), configured with strict port allowances:

Port Range	Protocol	Source	Purpose
22	TCP	0.0.0.0/0	SSH Administration via key pair (student-key.pem)
80	TCP	0.0.0.0/0	HTTP Web Traffic

Port Range	Protocol	Source	Purpose
8080	TCP	0.0.0.0/0	Spring Boot REST API Service

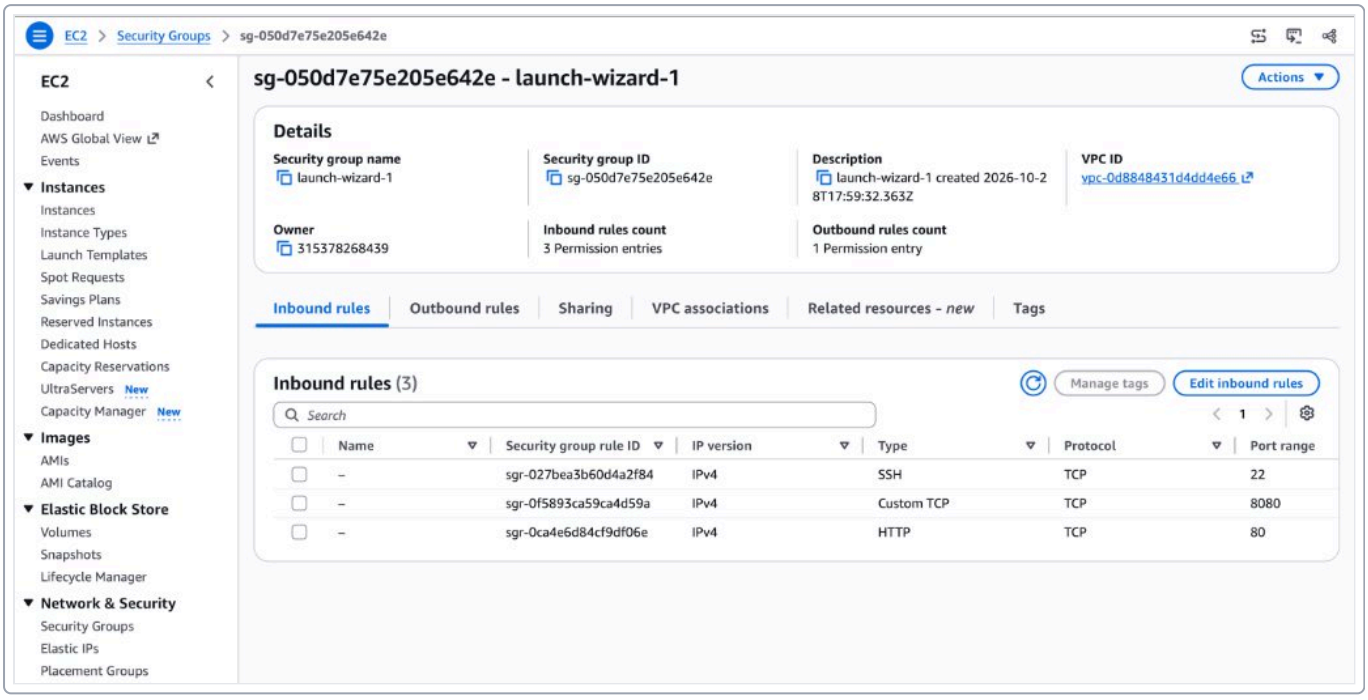


Figure 5.2: Inbound Firewall Security Group Rules on Amazon EC2 (Ports 22, 80, 8080).

5.3 Service Daemonization via Linux systemd

To ensure the application remains permanently active and automatically reboots if the virtual server restarts, the backend is managed by `systemd` at `/etc/systemd/system/student-manager.service`:

```
[Unit]
Description=Student Task & Assignment Manager Backend
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu
ExecStart=/usr/bin/java -jar /home/ubuntu/app.jar --spring.profiles.active=prod
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

6. DATABASE & PERSISTENCE TIER (AMAZON RDS)

The persistence tier utilizes **Amazon Relational Database Service (RDS)** for PostgreSQL. Decoupling storage from EC2 prevents data loss during instance termination and provides managed snapshot capabilities.

6.1 Managed Database Specifications

- **DB Identifier:** `student-manager-db`
- **Engine:** PostgreSQL (Version 18.3 / PostgreSQL License)
- **Instance Class:** `db.t4g.micro` (AWS Graviton2 Processor)
- **Status:** `Available`
- **Assigned AZ:** `ap-south-1c`
- **Database Endpoint:** `student-manager-db.ct8my864ardl.ap-south-1.rds.amazonaws.com`
- **Port:** 5432

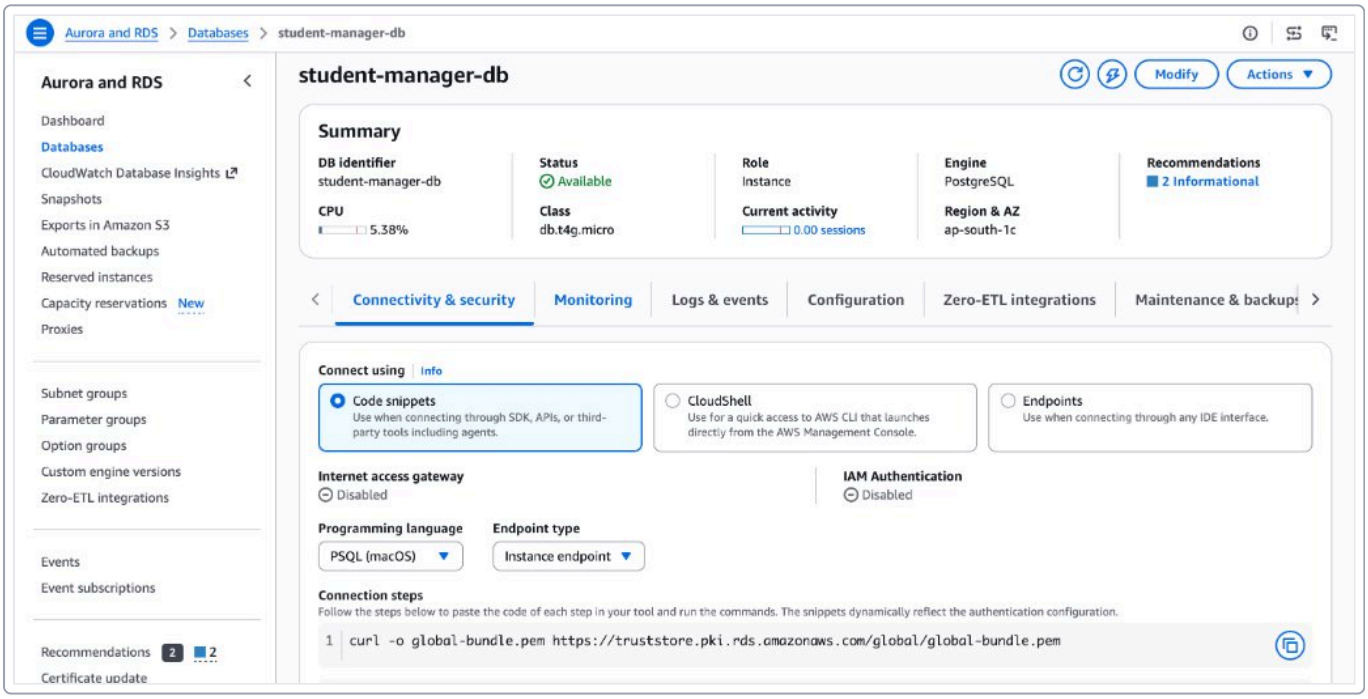


Figure 6.1: Amazon RDS PostgreSQL Instance Details and Available Status.

6.2 Storage and Encryption Parameters

The database instance is provisioned with 20 GiB of General Purpose SSD (`gp2`) storage with AWS KMS encryption enabled:

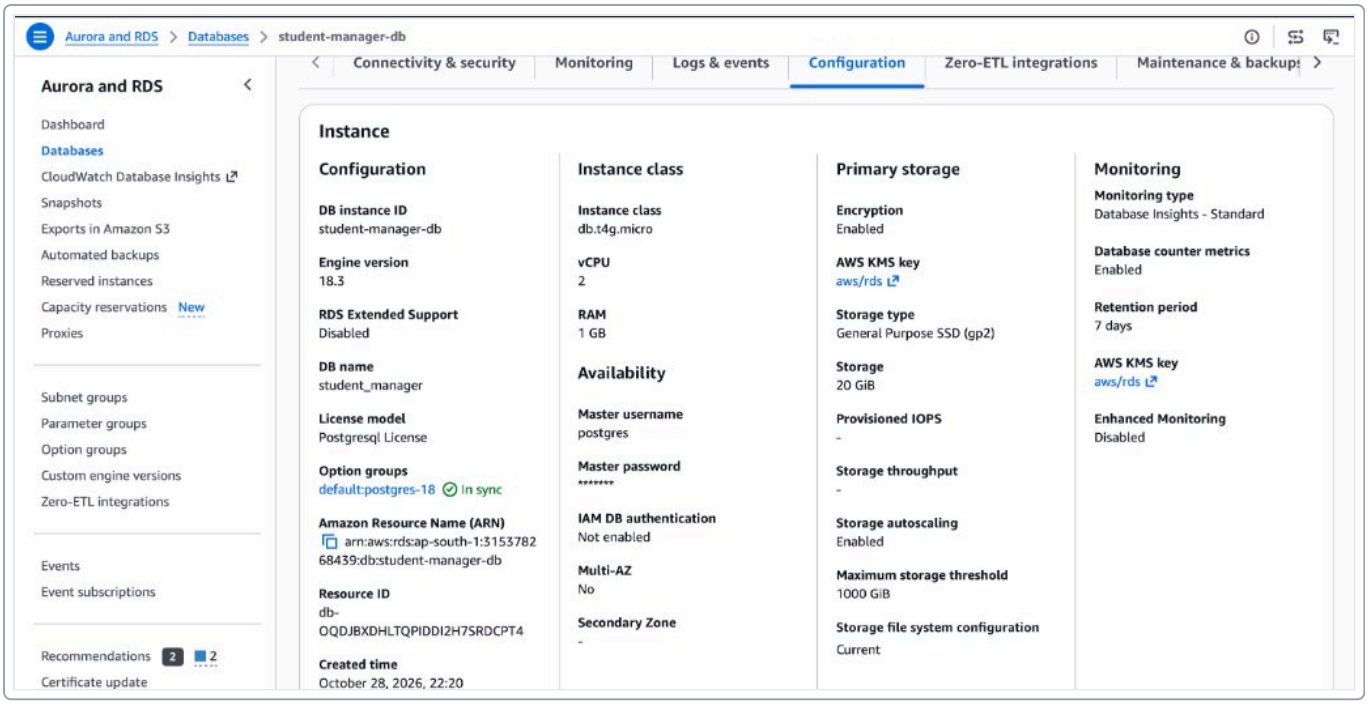


Figure 6.2: Amazon RDS Storage Parameters (20 GiB gp2, Database Name: student_manager).

6.3 Relational Schema Definition

The core relational entity is defined in PostgreSQL under the `assignments` table:

Column	Type	Constraints	Description
`id`	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique record identifier
`title`	VARCHAR(255)	NOT NULL	Assignment title
`subject`	VARCHAR(100)	NOT NULL	Course or subject name
`description`	TEXT	NULLABLE	Instructions, notes, and rubric details
`due_date`	DATE	NULLABLE	Target completion deadline
`priority`	VARCHAR(20)	NOT NULL, DEFAULT 'MEDIUM'	LOW, MEDIUM, HIGH
`status`	VARCHAR(20)	NOT NULL, DEFAULT 'PENDING'	PENDING, IN_PROGRESS, COMPLETED
`created_at`	TIMESTAMP	NOT NULL	System creation timestamp

7. IMPLEMENTATION & LIVE DEPLOYMENT VERIFICATION

7.1 Live Web Application Dashboard

The user interface provides real-time academic workload metrics and an interactive Kanban board:

- **AWS API Health Pill:** Displays a live pulsing indicator confirming continuous connectivity between S3 and the EC2 API.
- **Semester Velocity Progress Bar:** Dynamically recalculates the overall percentage of completed tasks.
- **Overdue Alert Banner:** Automatically warns students of tasks past their target submission date.
- **Kanban Status Workflow:** One-click status transitions (`Start →`, `Done ✓`, `Reopen ↺`) update PostgreSQL in real-time.

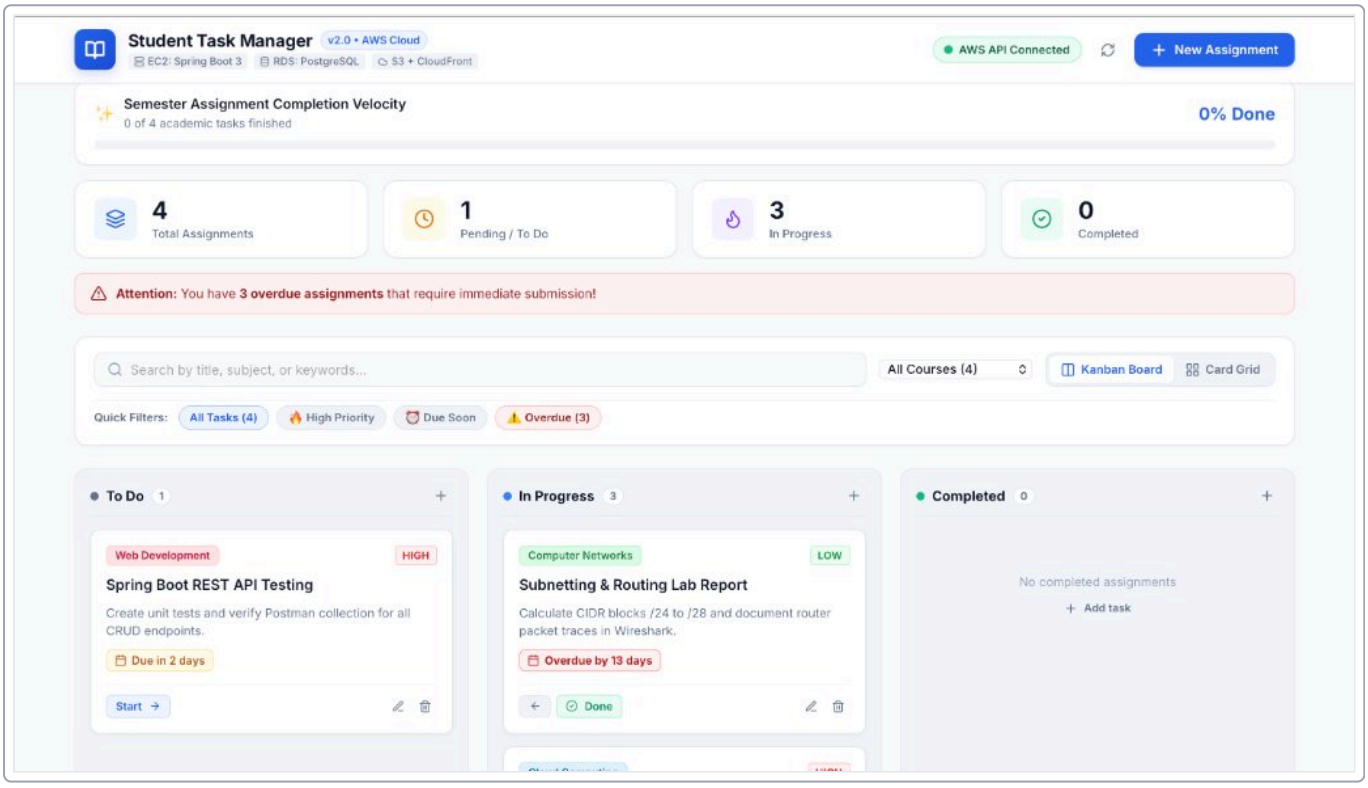


Figure 7.1: Live Application Dashboard with Kanban Workflow and Completion Velocity Meter.

7.2 Dynamic Task Creation Interface

The creation modal includes course quick-select pills and preset due-date calculators that synchronize with Amazon RDS via HTTP POST requests:

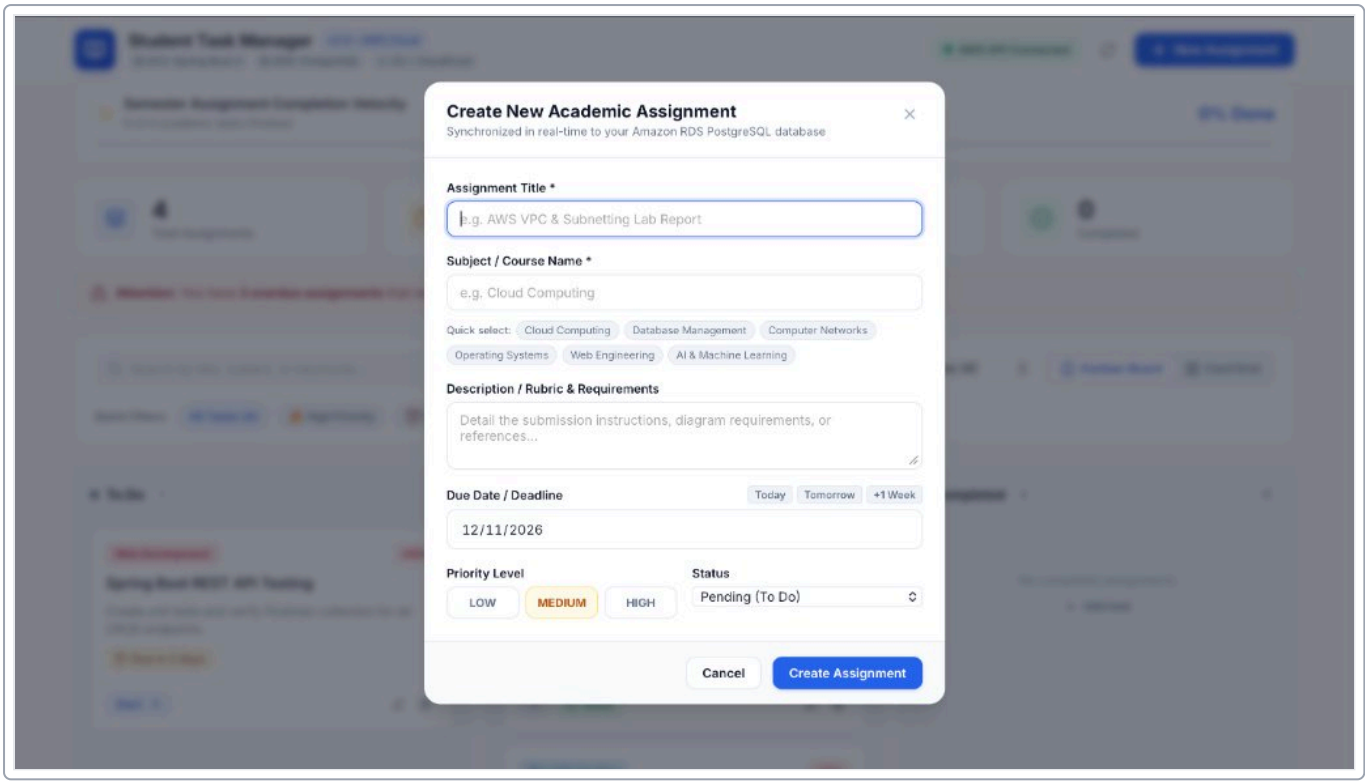


Figure 7.2: Assignment Creation Modal with Real-Time Persistence into Amazon RDS.

7.3 Backend REST API Health Verification

Independent verification of the application tier was conducted via HTTP GET requests to `/api/health` on the EC2 host:

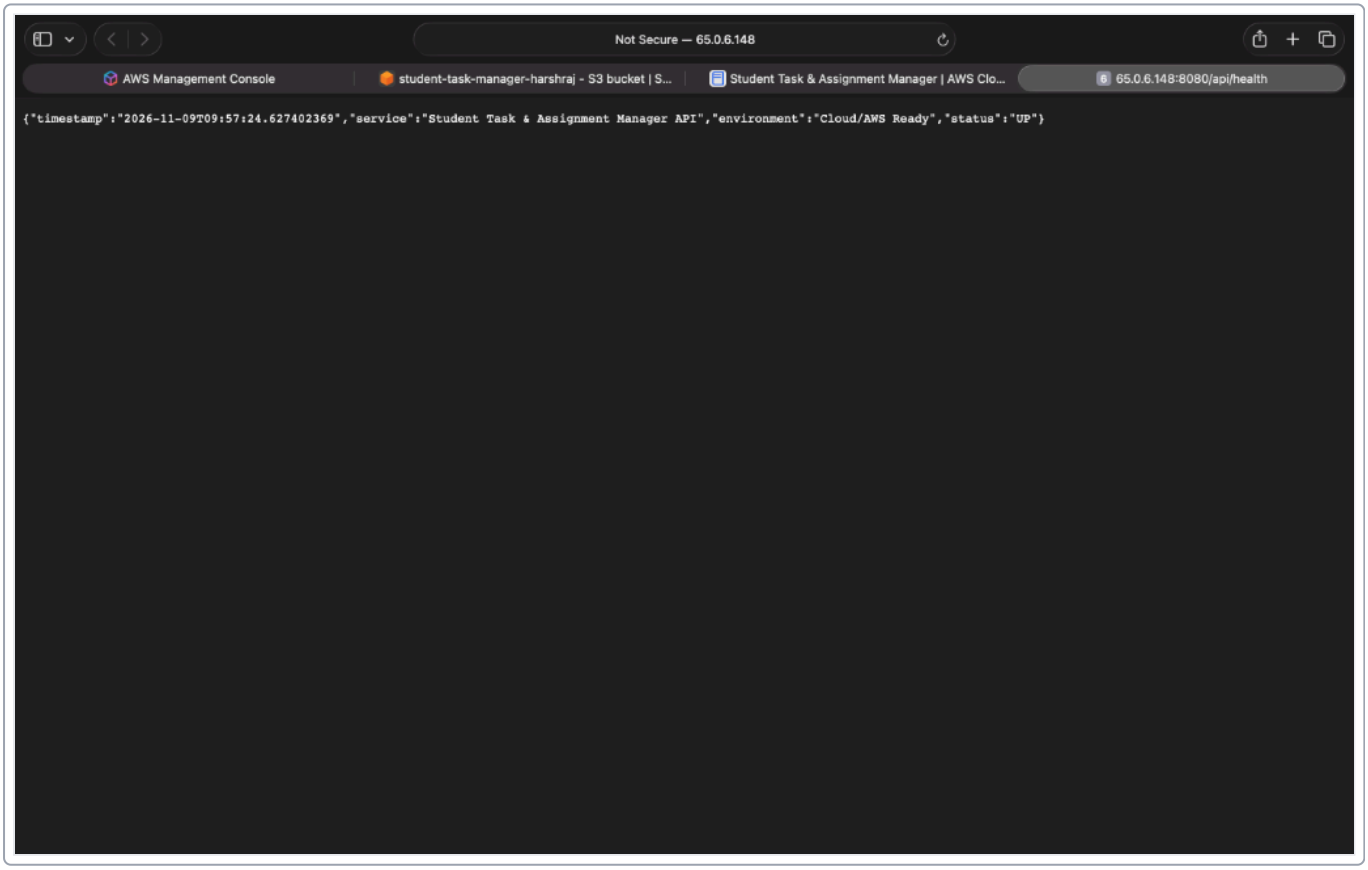


Figure 7.3: Backend REST API Health Check Response from EC2 (65.0.6.148:8080/api/health).

```
{
  "timestamp": "2026-11-09T09:57:24.627402369",
  "service": "Student Task & Assignment Manager API",
  "environment": "Cloud/AWS Ready",
  "status": "UP"
}
```


7.4 GitHub Version Control & Repository Deliverable

All source code, configuration files, and build scripts are version-controlled and hosted on GitHub:

- **Repository URL:** https://github.com/Harshraj2007/student-task-manager-aws

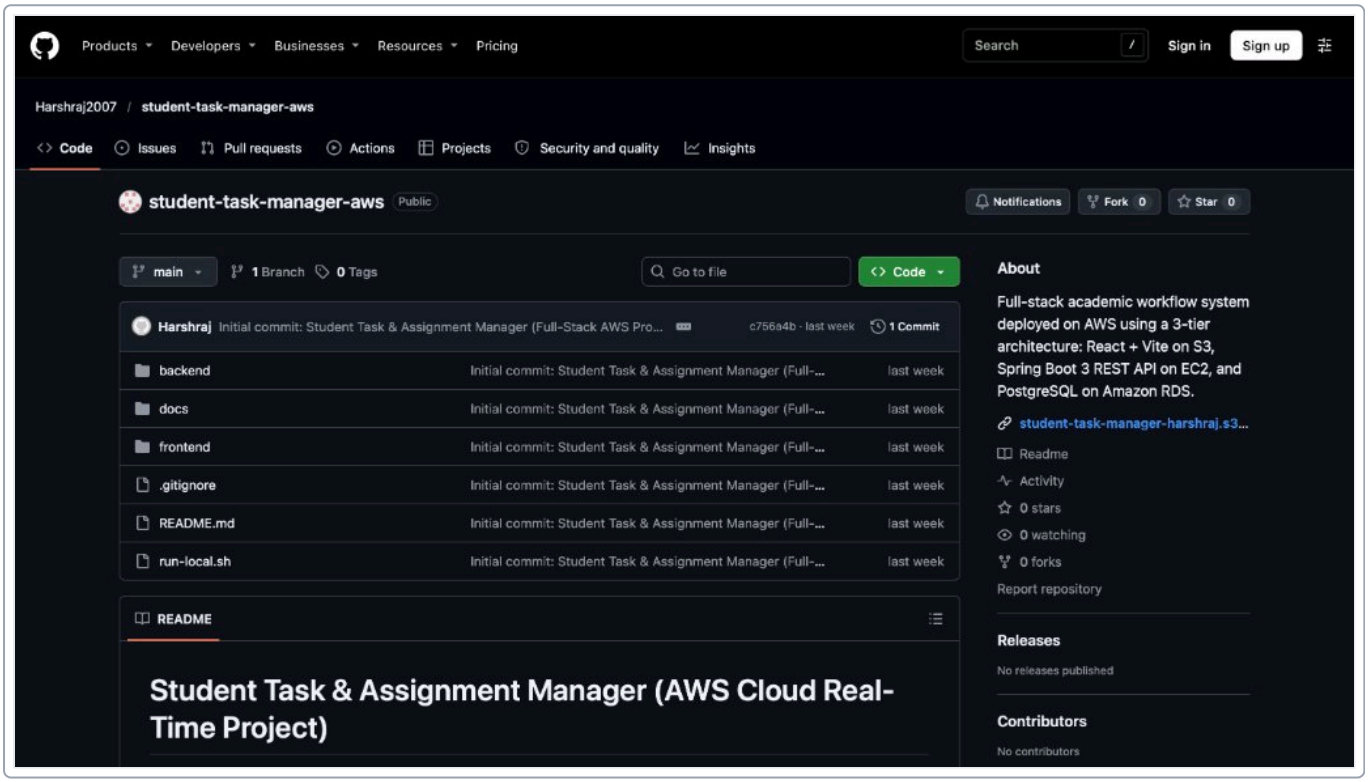


Figure 7.4: Public GitHub Repository Containing Project Source Code and Documentation.

8. AWS COST & FREE TIER ANALYSIS

To ensure financial sustainability, all provisioned resources strictly adhere to the **AWS 12-Month Free Tier** guidelines:

AWS Service	Allocated Resource	Monthly Free Tier Allowance	Actual Cost
Amazon EC2	1 × `t3.micro` Instance (Ubuntu 24.04)	750 Hours / Month	**$0.00**
Amazon RDS	1 × `db.t4g.micro` PostgreSQL Instance	750 Hours / Month	**$0.00**
RDS Storage	20 GiB General Purpose SSD (gp2)	20 GiB / Month	**$0.00**
Amazon S3	Static Assets (~1.2 MB)	5.0 GB Standard Storage	**$0.00**
Data Transfer	Outbound REST API & Web Responses	15 GB Outbound / Month	**$0.00**
		TOTAL MONTHLY EXPENDITURE:	**$0.00 (₹0.00)**

9. CONCLUSION & DELIVERABLES

The **Student Task & Assignment Manager** project successfully implements a modern full-stack web application hosted on an enterprise 3-tier AWS architecture for **CSE2025: Amazon Web Services**:

- **Decoupled Presentation Tier:** S3 serverless static hosting eliminates web serving overhead on compute nodes.
- **Resilient Compute Tier:** Spring Boot 3 running on EC2 is reinforced with Linux `systemd` service auto-restart daemons.
- **Managed Relational Persistence:** Amazon RDS PostgreSQL guarantees data integrity and high availability with automated backups.
- **Cost-Effective Engineering:** The solution provides full production capability while maintaining zero infrastructure cost.

Project Deliverable	Access Link / Location
Live Web Application	http://student-task-manager-harshraj.s3-website.ap-south-1.amazonaws.com/
Backend API Health Check	http://65.0.6.148:8080/api/health
Source Code (GitHub)	https://github.com/Harshraj2007/student-task-manager-aws